

SELF-HOSTING GUIDE

Deploy your own voice capture app

From an empty folder to an app on your iPhone — on a Windows PC, without owning a Mac.

Roughly 90 minutes of setup, most of it waiting for Apple.
Everything after that is one click.

You will need: a GitHub account, a credit card for Apple (\$99/year), and an Anthropic API key.

OVERVIEW

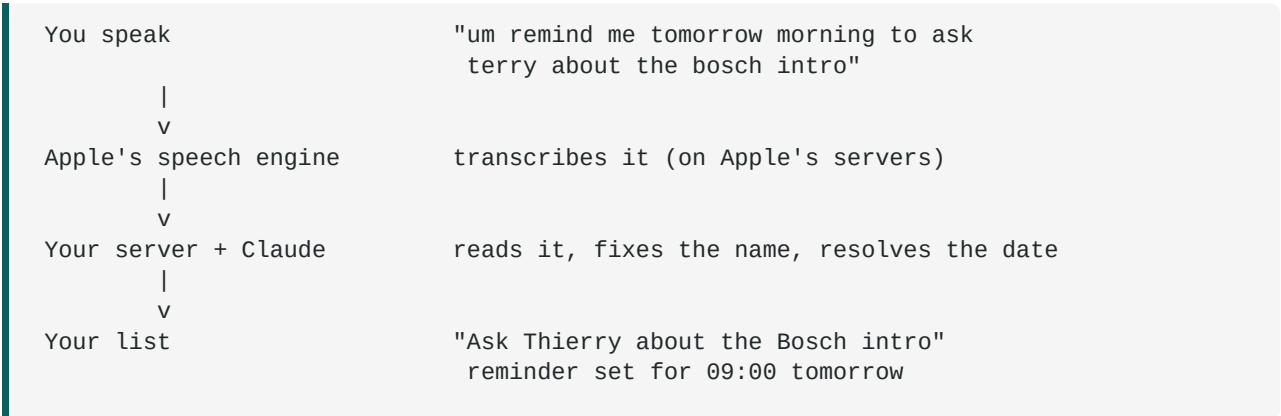
What you are building

This app catches a thought the moment you have one. You tap a microphone, say a sentence, and it becomes one clean line on a list. No forms, no categories, no decisions at the moment of capture.

Three separate things make that work, and you will set up each one:

Piece	What it does	Where it runs
The app	The screens on your iPhone. Records your voice, keeps your thoughts, reminds you.	Your phone
The server	Turns a rambling transcript into a clean thought, using Claude.	Railway
The build pipeline	Compiles the iOS app, signs it, uploads it to Apple.	GitHub Actions

How your voice becomes a thought



The important part: Claude sees your people and projects, so it corrects "terry" to "Thierry" and knows what "tomorrow morning" means. Without it, the app falls back to simple on-device parsing and your notes stay messy.

What it costs

Item	Cost	Required?
Apple Developer Program	\$99 / year	Yes, for any app on an iPhone
Anthropic API	Pay per use — pennies a day for personal use	Optional, but it is the whole point
Railway	Free tier is enough to start	Yes, to reach Claude
GitHub Actions	Free for public repositories	Yes

ONE HONEST WARNING

GitHub gives free macOS build minutes to public repositories only. If your repository is private, a release build costs roughly a dollar. Nothing else here has a surprise bill.

PART 1

The code on your machine

You need Node 22 or newer and Git. Check with:

```
node --version
git --version
```

Then get the code and start it:

```
git clone https://github.com/<your-account>/tasks.git
cd tasks
npm ci
npm run dev
```

Open the address it prints. You will see the desktop version. To see the phone app, narrow your browser window to under 768 pixels wide — the layout switches to the iPhone design, dark theme and all.

WHY NPM CI AND NOT NPM INSTALL

`npm ci` installs the exact versions recorded in `package-lock.json`. The build pipeline uses the same command, so what you test locally is what ships. If you ever see a build fail immediately on the server, an out-of-date lockfile is the first thing to check — commit it whenever you add a dependency.

Make it your own

Before you build anything, change the app identifier. It must be globally unique across the App Store and you cannot change it later without creating a new app.

Edit `capacitor.config.ts` and replace `com.moussazaghoud.hence` with your own reversed domain, for example `com.yourname.capture`. Then run:

```
npx cap sync ios
```

A Claude API key

1. Go to console.anthropic.com and sign in.
2. Open Settings, then API keys.
3. Create a key and copy it. You only see it once.
4. Add a small amount of credit — a few dollars lasts a long time for personal use.

KEEP THIS ONE SECRET

Unlike most values in this guide, an API key is a real credential. It never goes into the app itself, only onto your server. Anything inside an iPhone app can be extracted from the download.

To try it locally before deploying, create a file named `.env.local` in the project root:

```
ANTHROPIC_API_KEY=sk-ant-...
```

Restart `npm run dev`, speak something into the browser, and the captured line should come back clean rather than word-for-word.

The server

The app on your phone cannot call Claude directly — that would put your API key inside the download. Instead it asks a small server that you own.

- 1. Push your copy of the code to your own GitHub repository.
- 2. Go to railway.app and sign in with GitHub.
- 3. New Project, then Deploy from GitHub repo, and pick your repository.
- 4. Open the service, go to Variables, and add ANTHROPIC_API_KEY with the key from Part 2.
- 5. Go to Settings, then Networking, and click Generate Domain. Copy the address it gives you.

THE MISTAKE THAT COSTS AN AFTERNOON

If your host decides on its own how to build the project, it may conclude this is a plain website, serve the files, and never start the server. Everything looks fine — the page loads — but the part that calls Claude was never running. The repository includes a Dockerfile precisely to prevent that guesswork. Make sure no Output Directory is set in the service settings, and leave the start command empty.

Check it actually works

Open your Railway address in a browser and add /healthz to the end. You want to see exactly this:

```
{"ok":true,"voice":true}
```

What you see	What it means
<code>{"ok":true,"voice":true}</code>	Correct. The server is running and can see your Claude key.
<code>{"ok":true,"voice":false}</code>	The server runs but the key is missing or on the wrong service.
A web page instead of text	Your host is serving files, not running the server. See the warning above.

The Apple Developer Program

This is the slow step, so start it early. Everything after it waits on approval.

1. Turn on two-factor authentication for the Apple ID you will use. Enrolment refuses to continue without it.
2. Go to developer.apple.com/programs and click Enrol.
3. Choose Individual unless the app must be published under a company name. Organization enrolment needs a D-U-N-S number and takes one to three weeks; Individual is usually approved within a day.
4. Pay the \$99. Approval arrives by email.

USE AN APPLE ID YOU WILL KEEP

The account owns the app. Moving an app to a different Apple ID later is a support request, not a setting.

Once approved, note your Team ID: developer.apple.com/account, then Membership details. It is ten characters, something like A1B2C3D4E5.

The App Store Connect key

This key lets the build pipeline talk to Apple on your behalf, so you never sign anything by hand.

1. Go to appstoreconnect.apple.com, then Users and Access, then Integrations.
2. Choose App Store Connect API, then Team Keys, then the plus button.
3. Name it CI and give it the App Manager role.
4. Download the .p8 file. You get exactly one chance to download it.
5. Note the Key ID next to it, and the Issuer ID shown above the table.

If your Apple account is in French, the same path reads: Utilisateurs et accès, Intégrations, API App Store Connect, Clés d'équipe, and the role is Responsable d'app.

Turn the key file into one line

GitHub stores secrets as text, so the .p8 file has to be encoded. In PowerShell, from the folder where you saved it:

```
[Convert]::ToBase64String(  
    [IO.File]::ReadAllBytes("AuthKey_ABCD1234.p8")  
) | Set-Clipboard
```

That puts one long line on your clipboard. Paste it straight into GitHub in Part 7.

The certificates repository

Signing an iOS app needs a certificate. Rather than generating one by hand on a Mac you do not have, the pipeline creates it once and stores it encrypted in a private repository of yours.

1. On GitHub, create a new repository named something like app-certificates. Make it Private. Leave it empty.
2. Go to Settings, then Developer settings, then Personal access tokens, then Fine-grained tokens.
3. Create a token with access to only that one repository, and give it the Contents: Read and write permission.
4. Copy the token.

Now encode your username and token together, again in PowerShell:

```
[Convert]::ToBase64String(  
    [Text.Encoding]::UTF8.GetBytes("yourusername:github_pat_xxx")  
) | Set-Clipboard
```

INVENT A PASSPHRASE NOW

You will need a MATCH_PASSWORD in the next part. It encrypts your certificates. Save it in your password manager before you continue — if you lose it, you have to delete the certificates and start this part again.

Secrets and variables

In your main repository: Settings, then Secrets and variables, then Actions.

Secrets tab

Name	Value
ASC_KEY_ID	The Key ID from Part 5
ASC_ISSUER_ID	The Issuer ID from Part 5
ASC_KEY_P8	The long base64 line from Part 5
APPLE_TEAM_ID	Your ten-character Team ID
MATCH_GIT_URL	<code>https://github.com/<you>/app-certificates.git</code>
MATCH_GIT_BASIC_AUTHORIZATION	The base64 username:token from Part 6
MATCH_PASSWORD	The passphrase you invented

Variables tab

Name	Value
API_BASE_URL	Your Railway address, including <code>https://</code>
APP_NAME	The name of your app, if not Hence
APP_IDENTIFIER	Your bundle identifier, if you changed it

INCLUDE THE HTTPS://

A bare address like `myapp.up.railway.app` is read as a folder name rather than a website, so the app quietly falls back to on-device analysis and looks like the AI simply is not very good. The code now repairs this, but type it correctly anyway.

SECRETS ARE READ WHEN A RUN STARTS

If you start a build and then add a missing secret, that run still fails. Start a new one.

The App ID and the app record

Two things must exist on Apple's side before a build can be uploaded. One is automated; the other is not.

Register the App ID

In your repository: Actions, then iOS release, then Run workflow, and choose bootstrap. This registers your bundle identifier with Apple. It takes under a minute.

Create the app record by hand

Apple's API cannot create an app record. There is no endpoint for it, with any key or any role. This step is done once in a browser and cannot be automated.

1. Go to appstoreconnect.apple.com/apps and click the plus button, then New App.
2. Platform: iOS.
3. Name: your app name. It must be unique across the entire App Store, so have a second choice ready.
4. Primary Language: whatever you prefer.
5. Bundle ID: pick yours from the list. If it is not there, the bootstrap step above did not finish.
6. SKU: any unique string, for example capture-001. Nobody sees it.
7. User Access: Full Access, then Create.

Your first build

Actions, then iOS release, then Run workflow, leaving the lane on beta.

Six to fifteen minutes. On the first run the pipeline also creates your distribution certificate and stores it in the private repository from Part 6; later runs reuse it.

What happens, in order:

- The web app is built, with your Railway address baked in.
- It is copied into the iOS project.
- The certificate and provisioning profile are fetched, or created the first time.
- Xcode archives and signs the app on a rented Mac.
- The result is uploaded to TestFlight.

THE VERSION AND BUILD NUMBER

The version shown to users comes from `package.json`. The build number is the workflow run number, so it always increases and always points back at a specific run. Apple rejects a build number it has seen before, so never re-run an old workflow — start a new one.

On your iPhone

1. In App Store Connect, open your app and go to the TestFlight tab.
2. The build shows as Processing for five to fifteen minutes. Wait for it to become a version number.
3. If you see Missing Compliance, click it and answer No to the encryption question. The app uses only standard HTTPS.
4. Under Internal Testing, create a group, add yourself, and select the build. Tick automatic distribution so future builds arrive on their own.
5. On the phone, install TestFlight from the App Store, sign in with the same Apple ID, and your app is there.

Check it end to end

Tap the microphone and say something deliberately messy, with a name and a vague time in it:

```
"um remind me tomorrow morning to ask terry about the bosch intro"
```

You should get back a single clean line, the name spelled correctly, and a reminder set for nine in the morning. The small message at the bottom of the screen tells you which engine read it:

Message	Meaning
Captured	Claude read it. Everything is connected.
Captured · on-device	The server was not reachable. Check Part 3 and the <code>API_BASE_URL</code> variable.

EXPECT TWO PERMISSION PROMPTS

The first time you tap the microphone, iOS asks separately for Speech Recognition and for the Microphone. Both are required. If you get no prompt at all and an error instead, the app was built without the speech component — that is a build problem, not a device problem.

Optional: your work calendar

Turning a thought into a calendar event works out of the box through the share sheet, which needs no setup and no permissions. If you would rather have events created directly in a Microsoft 365 calendar, register the app with your organisation:

1. Go to entra.microsoft.com, then App registrations, then New registration.
2. Name it, and choose the account types that match your organisation.
3. Under Redirect URI, choose the iOS / macOS platform and enter your bundle identifier. It generates a URI of the form `msauth.<your bundle id>://auth`.
4. Under Authentication, set Allow public client flows to Yes.
5. Under API permissions, add the delegated Microsoft Graph permissions `Calendars.ReadWrite` and `offline_access`.
6. Copy the Application (client) ID and the Directory (tenant) ID into your repository variables.

THIS MAY NOT BE YOURS TO DECIDE

Large organisations often disable user consent for new applications. If the permission status stays at Not granted and the consent button is greyed out, your IT administrator has to approve it. The share-sheet route keeps working in the meantime.

NEVER PUT A CLIENT SECRET IN THE APP

A phone uses PKCE, which proves the request came from the same app that started it, without storing a secret. If you are reusing a registration built for a server, take only the client ID and tenant ID from it and leave the secret where it is.

When something fails

These are real failures from a real first-time setup, with what each one actually meant.

What you see	What it really is
Build fails in under ten seconds	It failed before doing any work. Usually the lockfile is missing or out of date, or the host guessed the wrong kind of project.
The resource apps does not allow CREATE	Apple has no API for creating an app record. Do Part 8 by hand.
Could not find option api_key	An older tool that authenticates with an Apple ID rather than an API key. The pipeline in this repository does not use it.
Info.plist did not contain CFBundleShortVersionString	An empty version reached the build. An unset workflow input arrives as an empty string, which is not the same as unset.
Missing secret, though you added it	Secrets are read when a run starts. Start a new run.
POST to your server returns 405	Your host is serving files rather than running the server.
/healthz returns a web page	Same cause as above.
Voice fails with no permission prompt	Nothing native ran. The speech component is missing from the build, or not registered with the web layer.
Notes come out word for word	Claude is not being reached. Check /healthz, then the API_BASE_URL variable, including its https://.
The keyboard covers the text field	The keyboard resize mode is set to body rather than native.

Where your data goes

Worth knowing, and worth telling anyone you share the app with.

What	Where it goes
Your thoughts	Stored on your phone only. Deleting the app deletes them. Export before you do.
Your voice	Transcribed by Apple, usually on Apple's servers. It is not stored by this app.
The transcript	Sent to your own server, which forwards it to Anthropic for analysis. Not stored by either.
Your Claude API key	On your server only. Never inside the app.
Reminders	Local notifications on the phone. They work with the app closed, and are not synced anywhere.

What the app deliberately does not do

- It does not listen in the background. iOS does not permit it, and nothing here pretends otherwise.
- It does not sync between devices. Your phone and your laptop each keep their own list.
- It has no accounts, no analytics and no tracking.
- It requests no camera or photo access, which also keeps App Review simple.

IF YOU PUBLISH TO THE APP STORE

You will be asked about data collection. Answer honestly: the app collects User Content, for App Functionality, not linked to an identity. If you later add accounts or syncing, that answer must change.